Assignment: Bank Account Management System using C++

Problem Statement

Create a class named BankAccount to represent a bank account. Each account should store the account holder's name and current balance. The program must allow users to perform basic banking operations such as depositing money, withdrawing money, and applying interest to the account balance.

The system should also maintain information that is common to all bank accounts, such as the total number of accounts created and a shared interest rate, using static data members and static member functions.

Requirements

1. Use a class to model a bank account.
2. Use private data members to store:
    - Account holder name
    - Account balance
3. Implement a constructor to initialize account details and increment the total number of accounts.
4. Provide member functions to:
    - Deposit a valid amount into the account
    - Withdraw an amount with proper validation
    - Apply interest to the account balance
    - Display account details in a formatted manner
5. Use static data members to:
    - Store a common interest rate for all accounts
    - Keep track of the total number of bank accounts created
    - Keep track of the total money / balance in the bank
6. Use static member functions to:
    - Display the total number of accounts
    - Display the total balance in the bank

Demonstrate the working of the program by creating at least two bank account objects and performing various operations on them.

Assignment: Student Result Management System using C++

Problem Statement

Design a Student class to store student details and marks in three subjects.
The program should calculate total marks, assign grades, and maintain shared information such as the total number of students and the student with the highest total marks.

Requirements

1. Create a Student class with private data members:
    - Student name
    - Roll number
    - Class/Section
    - Marks in three subjects (individual variables)
2. Implement:
    - A constructor to initialize student details and marks
    - A member function to calculate total marks
    - A member function to calculate grade based on total marks
    - A member function to update marks (Note: While updating student marks, the updated marks for **all three subjects must be provided at the same time**.
      This ensures that the **total marks and grade are recalculated correctly)**
    - A member function to display student details, total marks, and grade
3. Use **static data members** to:
    - Store the total number of students
    - Store the highest total marks
    - Store the name of the student with the highest marks
4. Use **static member functions** to:
    - Display total number of students
    - Display the highest scorer details (name and marks)
5. Demonstrate the program by creating multiple student (at least 3) objects and performing updates.

Assignment: Demonstration of Compile-Time Polymorphism using Function Overloading

Problem Statement

Design a C++ program to demonstrate **function overloading** by creating a class that performs addition operations on different data types and different numbers of operands. The same function name should be used to add integers, floating-point numbers, double values, and strings.

Requirements

1. Create a class named AddDemo.
2. Implement overloaded add() functions to:
    - Add **two integers**
    - Add **three integers**
    - Add **two floats**
    - Add **three floats**
    - Add **two doubles**
    - Add **three doubles**
    - Concatenate **two strings**
3. Ensure that:
    - All functions use the **same function name**
    - The functions differ only in **number and/or type of parameters**
4. Demonstrate the working of each overloaded function in the main() function.